

Smart Pointer

Accedere alla memoria in modo controllato

Lezione 18
Corso 1



Operazioni sui puntatori

- Ogni valore manipolato da un programma è memorizzato nello spazio di indirizzamento del processo
 - L'operatore `&` (e `&mut`, in Rust) permette, in C, C++ e Rust, di ottenere l'indirizzo del primo byte in cui è memorizzato
 - Nel caso di Rust, tale operatore attiva il borrow checker che vigila sull'uso che viene fatto dell'indirizzo ottenuto, imponendo tutti i vincoli di sanità necessari a fornire le garanzie date dal modello del linguaggio
- L'operazione duale, detta dereferenza (dereferencing) o risoluzione del riferimento, trasforma un indirizzo nel corrispondente valore puntato
 - Si esprime con l'operatore `*` (e `->` in C e C++ o `.` in Rust)
 - Quando viene applicato ad un puntatore nativo o ad un riferimento Rust, il compilatore dà accesso al dato puntato

gli smart pointer permettono di fare delle operazioni aggiuntive che possono bypassare le regole del borrow checker

Operazioni sui puntatori

- Sia Rust che C++ permettono di ridefinire il comportamento degli operatori del linguaggio per tipi arbitrari
 - Entrambi permettono inoltre di definire tipi generici, che possono essere espansi in una molteplicità di tipi concreti, in funzione di come vengono utilizzati
- Questi meccanismi, applicati agli operatori di dereferenza abilitano la definizione di tipi che “sembrano” puntatori (dal punto di vista sintattico) ma che hanno ulteriori caratteristiche
 - Garanzia di inizializzazione e rilascio
 - Conteggio dei riferimenti
 - Accesso esclusivo con attesa
 - ...
- Ciò ha portato all'introduzione del concetto di “smart pointer”
 - E alla sua diffusione sia nelle librerie standard C++, dove l'uso dei puntatori nativi è causa di errori frequenti...
 - ...sia nelle librerie standard Rust, che ne hanno abbracciato l'idea per rappresentare puntatori che possiedono i dati a cui puntano (in contrapposizione ai riferimenti, che godono del solo prestito)

Uso dei puntatori

- Tramite l'uso di puntatori è possibile costruire strutture dati dinamiche (dalla topologia non prevedibile a priori e/o variabile nel tempo) come grafi, alberi, liste
 - Se questo, da un lato, offre grandi libertà al programmatore, dall'altro lo espone ad una serie di problemi legati alla difficoltà di dedurre la correttezza del codice che lo manipola
- Le regole restrittive imposte dal borrow checker di Rust impediscono, con l'uso di soli riferimenti, la creazione di strutture cicliche
 - Ogni valore in Rust è parte di un solo albero la cui radice è contenuta in una qualche variabile
 - Questa minore capacità espressiva abilita, però, analisi più approfondite ed è alla base delle garanzie di sanità offerte dal linguaggio
- Attraverso l'uso di smart pointer come `Rc<T>` e `Arc<T>`, è possibile avere più possessori di uno stesso valore
 - Smart pointer come `std::rc::Weak` e `std::sync::Weak` offrono invece la possibilità di avere strutture cicliche, nel rispetto di alcune restrizioni

Smart pointer in Rust

- Rust offre una varietà maggiore di smart pointer rispetto al C++, allo scopo di coprire ulteriori casi e definire ottimizzazioni possibili nel caso specifico di programmi puramente sequenziali piuttosto che di programmi concorrenti
 - Alcuni di questi ricalcano abbastanza fedelmente le astrazioni offerte dal C++ (`Box<T>`, `Rc<T>`, `Arc<T>`, `Weak<T>`)
 - Altri sono peculiari del modello introdotto dal linguaggio (`Cell<T>`, `RefCell<T>`, `Cow<T>`, `Mutex<T>`, `RwLock<T>`)
- In generale, sono realizzati mediante struct che contengono le necessarie informazioni e che implementano i tratti `Deref` e `DerefMut`
 - Quando il compilatore incontra l'espressione `*ptr` (dove il tipo di `ptr` implementa tali tratti) la trasforma in `* ptr.deref()` o `* ptr.deref_mut()` a seconda dei casi

I tratti Deref e DerefMut

```
trait Deref {  
    type Target: ?Sized;  
    fn deref(&self) -> &Self::Target;  
}  
  
trait DerefMut: Deref {  
    fn deref_mut(&mut self) -> &mut Self::Target;  
}
```

std::Box<T>

- Struttura che incapsula un puntatore ad un blocco allocato dinamicamente sullo heap all'atto della sua costruzione (tramite il metodo **Box::new(t)**)
 - Il dato puntato è **posseduto** (unicamente) da Box: quando la struttura esce dal proprio scope sintattico, il blocco sullo heap viene rilasciato automaticamente, grazie all'implementazione del tratto **Drop**
 - E' possibile anticipare il rilascio del blocco, invocando la funzione **drop(b)**
- Se la struttura viene mossa in un'altra variabile (o ritornata da una funzione), il possesso del puntatore passa alla destinazione che diventa responsabile del suo rilascio
 - Questo rende possibile ottenere cicli di vita che si estendono oltre la durata della funzione in cui il dato è stato creato
- Il tipo T può avere una dimensione non nota in fase di compilazione (ovvero non implementare il tratto **Sized**)
 - In questo caso, l'oggetto di tipo **Box<T>** si trasforma in un fat pointer formato da un puntatore seguito da un intero di dimensione **usize** contenente la lunghezza del dato puntato
 - Analogamente, se al posto del tipo concreto T si indica un oggetto-tratto (**dyn Trait**), si ha un fat pointer composto da due puntatori: quello al dato sullo heap e quello a vtable del tratto

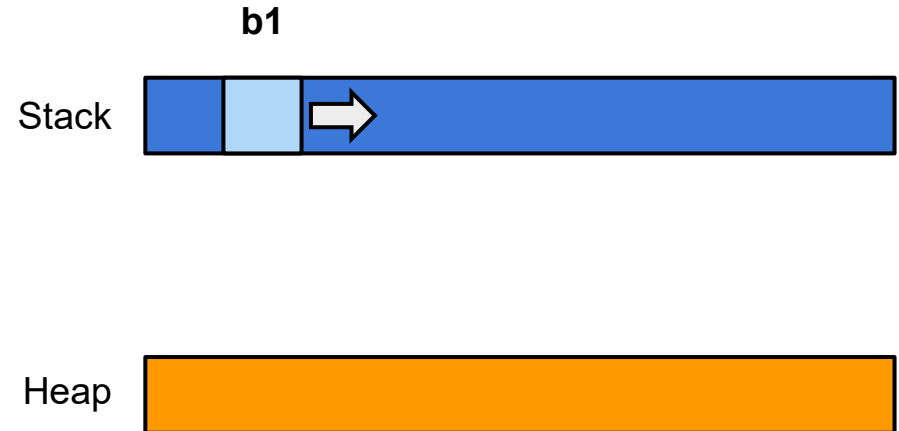
il tipo T nel box non deve necessariamente essere di tipo Sized. Se conosco la dimensione del dato allora la variabile di tipo Box corrisponde ad un puntatore semplice.
Al contrario, se il dato contenuto nel box non è Sized (ad esempio un array), occorre implementare la variabile Box con un fat pointer, che contiene il puntatore e la lunghezza

std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```

IMPORTANTE

se un dato è memorizzato nello heap, la variabile può durare per un tempo maggiore della vita della funzione in cui è stata allocata, perchè quella variabile può essere ceduta come possesso, attraverso il valore di ritorno di una funzione. Se la funzione ritorna un Box, la funzione trasferisce al chiamante la variabile e il dato non viene perso ma viene trasferito al chiamante



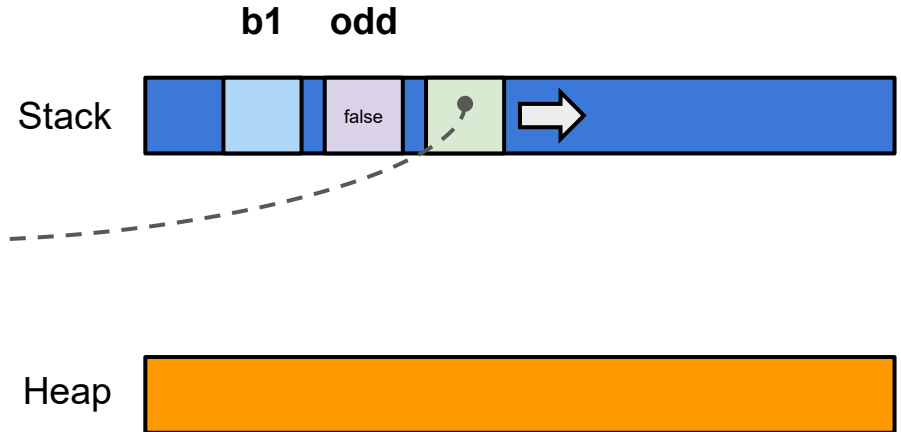
il box può contenere un Dyn; in questo caso la variabile box contiene due puntatori

- 1) uno al dato
- 2) uno alla VTABLE

Salta a slide 25

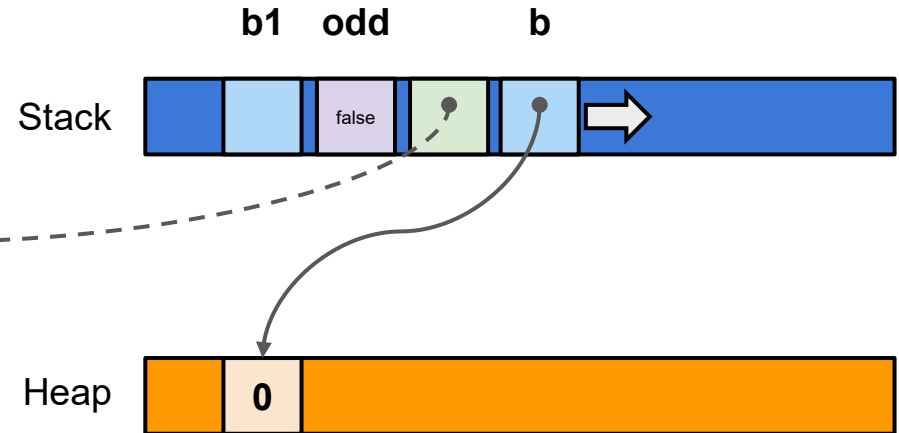
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



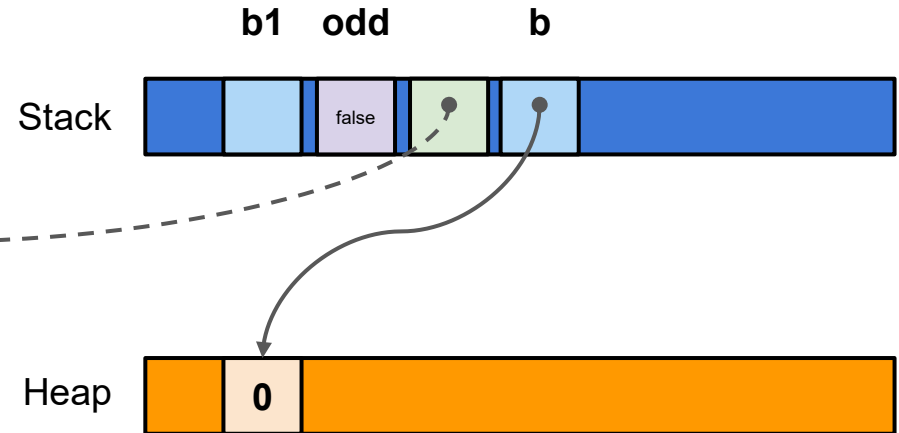
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

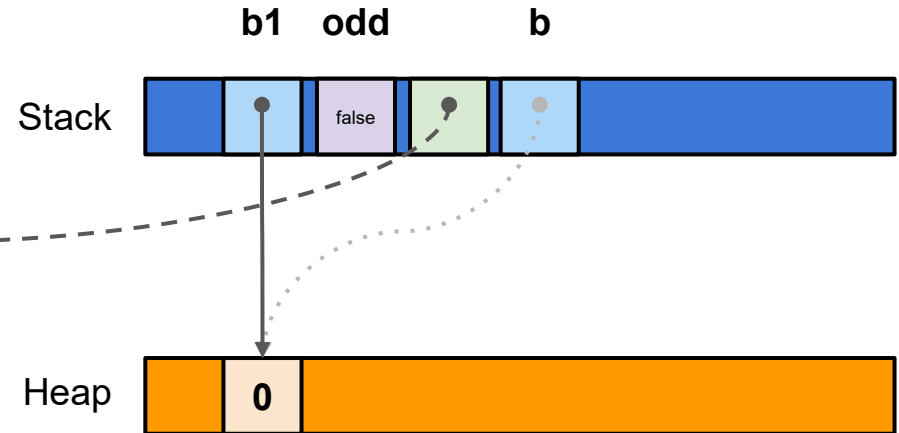
```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}
```

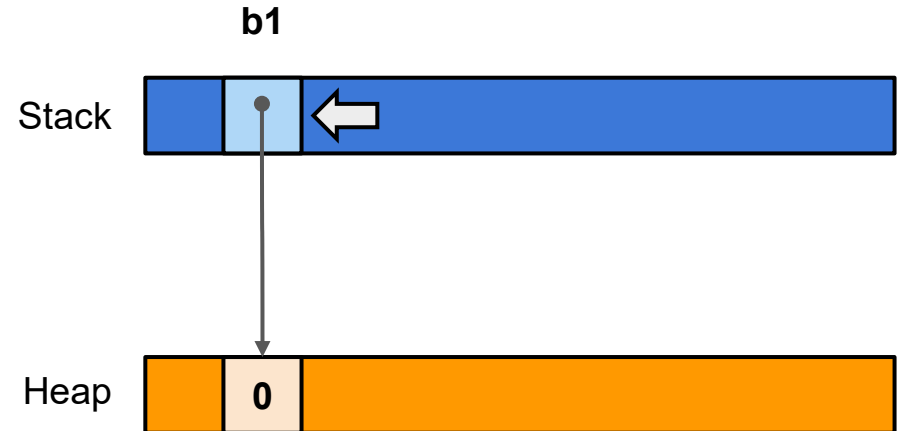
```
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

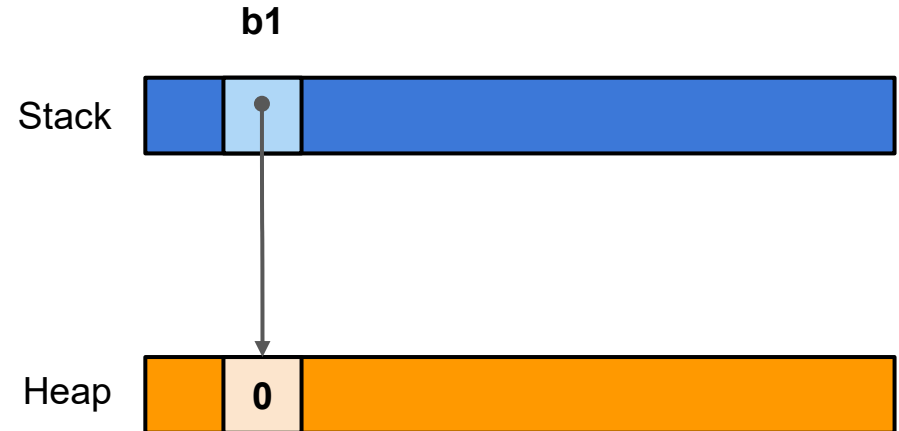
```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}
```

```
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



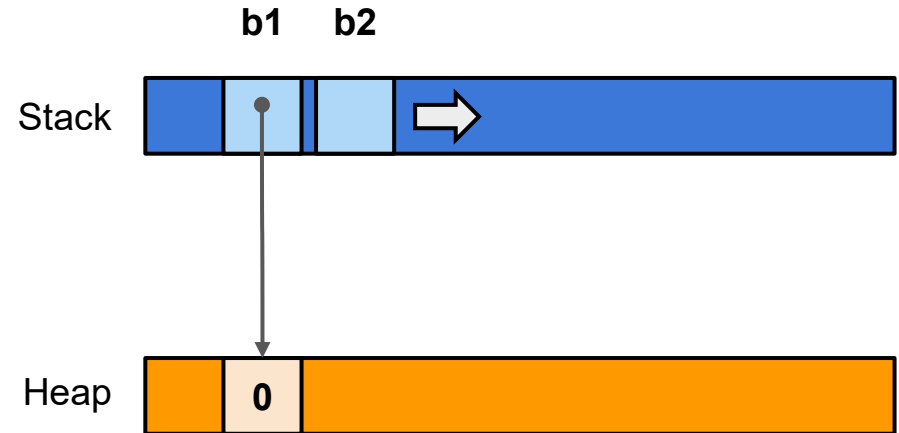
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b2);  
    println!("b2: {}", b2);  
}
```



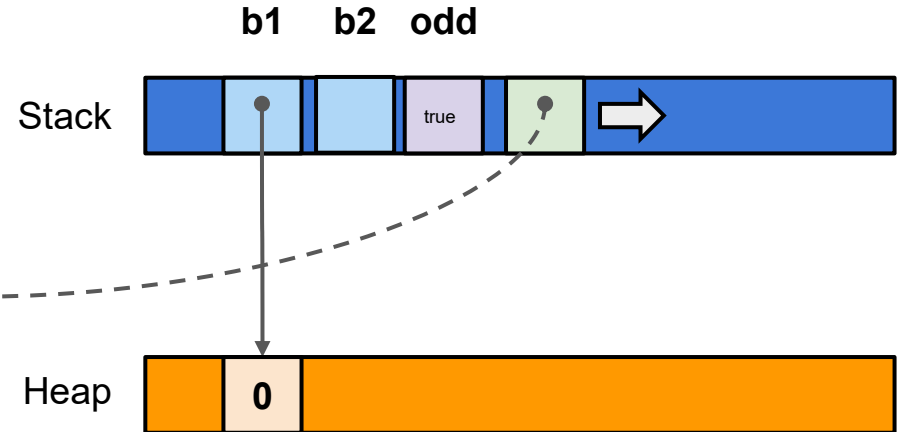
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



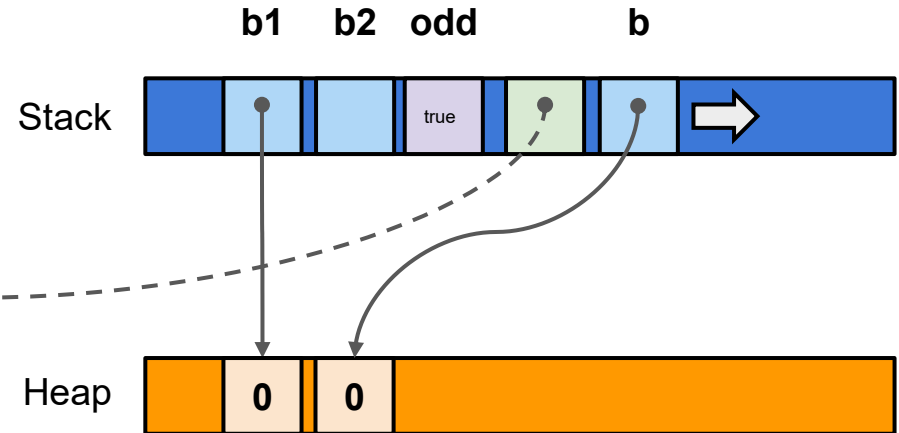
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



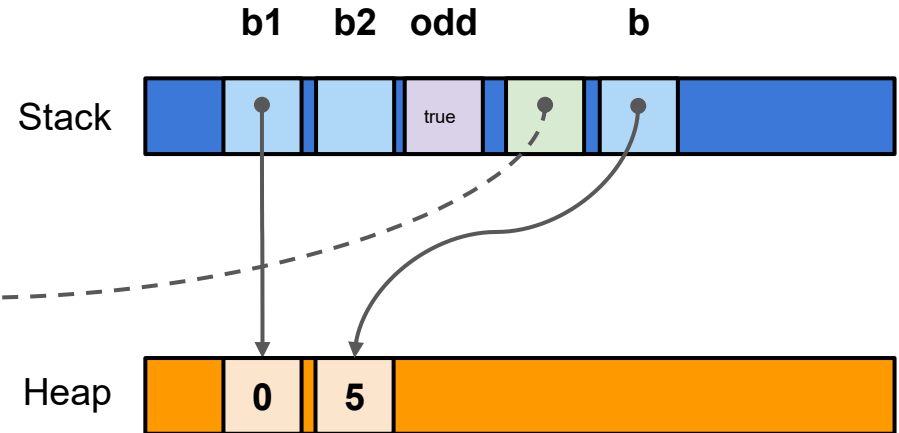
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

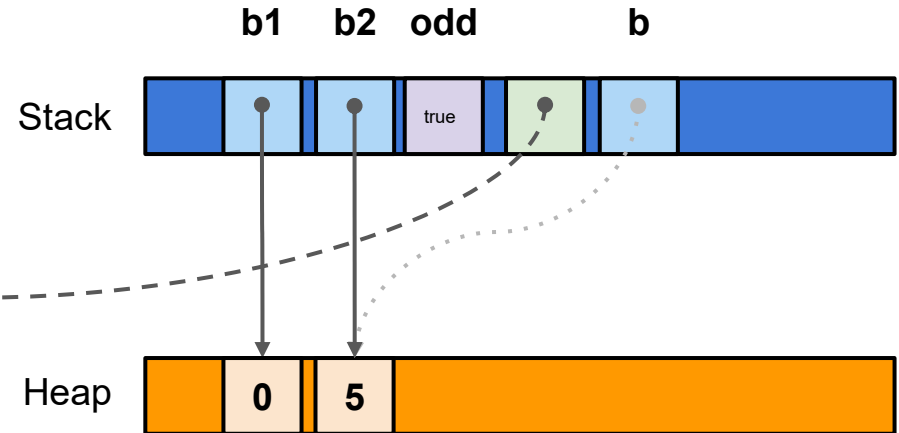
```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}
```

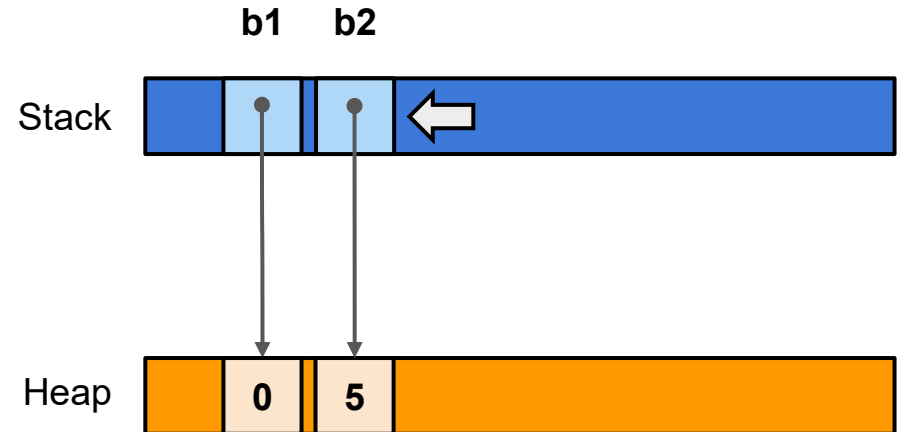
```
fn main() {  
    let b1 = produce(false);  
    println!("{}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("{}", b2);  
}
```



std::Box<T>

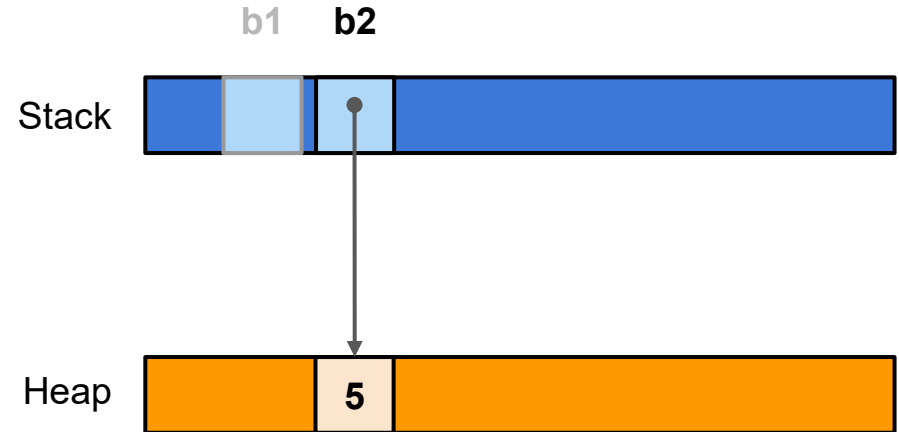
```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}
```

```
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



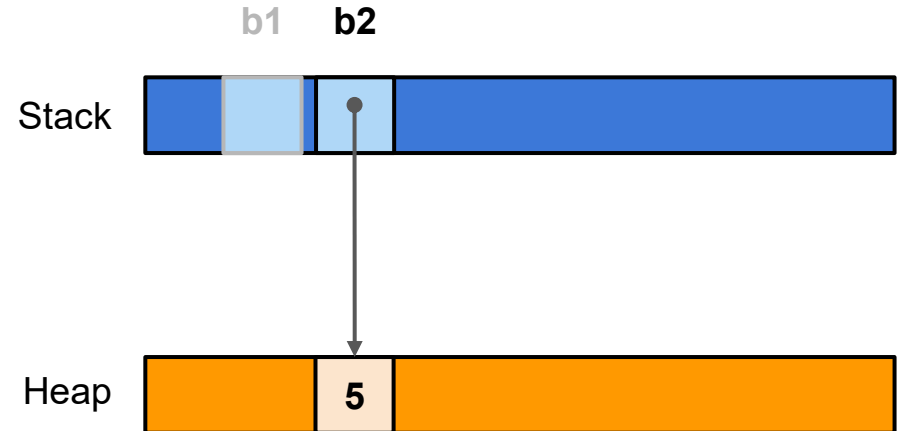
std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



std::Box<T>

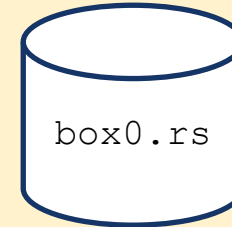
```
fn produce(odd: bool) -> Box<i32> {  
    let mut b = Box::new(0);  
    if odd { *b = 5; }  
    return b;  
}  
  
fn main() {  
    let b1 = produce(false);  
    println!("b1: {}", b1);  
    let b2 = produce(true);  
    drop(b1);  
    println!("b2: {}", b2);  
}
```



Allocazione nello heap

se faccio una lettura o una scrittura del box, devo usare l'asterisco. Al contrario, nella println() non occorre

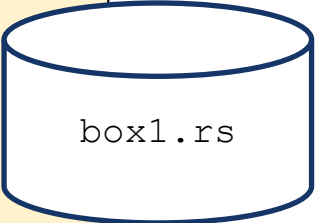
```
fn main() {  
    // Allocazione di un intero sulla heap con Box.  
    let mut numero = Box::new(10);  
  
    println!("Valore iniziale: {}", numero);  
  
    // Modifica del valore all'interno del Box.  
    *numero = 20;  
  
    println!("Valore modificato: {}", numero);  
}
```



Trasferimento di ownership

- I box facilitano il trasferimento di proprietà di oggetti grandi senza copiarli

```
struct LargeStruct {  
    // struttura con molti dati  
    data: [u8; 1024],  
}  
  
fn process_data(boxed_data: Box<LargeStruct>) {  
    // La proprietà viene trasferita qui senza copiare i 1024 byte  
    // Si opera direttamente sul puntatore  
    println!("Primo byte: {}", boxed_data.data[0]);  
}  
  
fn main() {  
    let large_data = Box::new(LargeStruct { data: [0; 1024] });  
    process_data(large_data); // Trasferimento efficiente  
}
```

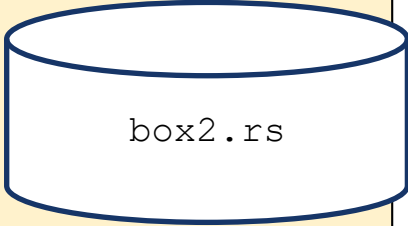


box1.rs

Implementazione di Tratti Oggetto

```
trait Animal {  
    fn make_sound(&self) -> String;  
}  
struct Dog;  
impl Animal for Dog {  
    fn make_sound(&self) -> String { "Bau!".to_string() }  
}  
struct Cat;  
impl Animal for Cat {  
    fn make_sound(&self) -> String { "Miao!".to_string() }  
}  
fn main() {  
    // Collezione di diversi tipi che implementano Animal  
    let animals: Vec<Box<dyn Animal>> = vec![Box::new(Dog), Box::new(Cat)];  
    for animal in animals {  
        println!("L'animale fa: {}", animal.make_sound());  
    }  
}
```

chat rust analizzo programmi singoli, command f “problema classico”



box2.rs

Dati con dimensione sconosciuta a compile-time

```
use std::io;

fn main() {
    println!("Inserisci la dimensione (in byte) del dato puntato dal Box:");

    let mut input_line = String::new();
    io::stdin()
        .read_line(&mut input_line)
        .expect("Errore nella lettura dell'input.");

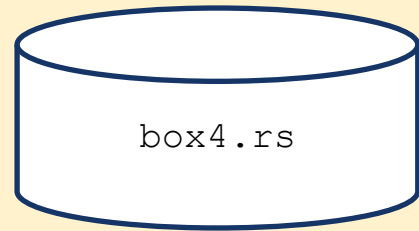
    let num_bytes: usize = input_line.trim().parse().unwrap();

    // Alloca un Vec<u8> con la dimensione specificata.
    let buffer: Vec<u8> = vec![0; num_bytes];

    // Converti il Vec in un Box<[u8]> per avere un puntatore a un blocco di memoria di dimensione
    // fissa sull'heap

    let boxed_slice: Box<[u8]> = buffer.into_boxed_slice();

    println!("Box<[u8]> allocato con successo. Dimensione: {} byte.", boxed_slice.len());
}
```



Che cosa memorizza
boxed_slice ?

Tipi Ricorsivi

- A titolo di esempio consideriamo la seguente struttura dati chiamata *Cons List* (*construct list*), struttura dati che deriva dal mondo LISP

```
enum List {  
    Cons(i32, List),  
    Nil,  
}
```

```
(1, (2, (3, Nil)))
```

- Ogni elemento (cella) della lista è composta da due parti:
 - Il valore dell'elemento corrente
 - Un puntatore alla cella successiva o a un valore terminale.

```
enum List {
    Cons(i32, List),
    Nil,
}
```



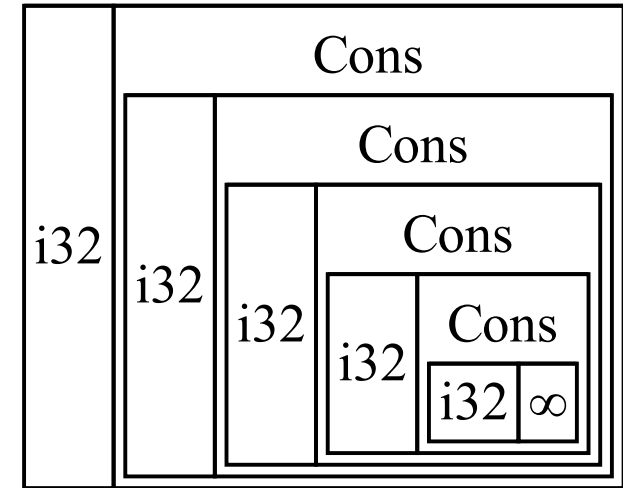
```
fn main() {
    let list = Cons(1, Cons(2, Cons(3, Nil)));
}
```



```
error[E0072]: recursive type `List` has infinite size
--> src\main.rs:1:1
```

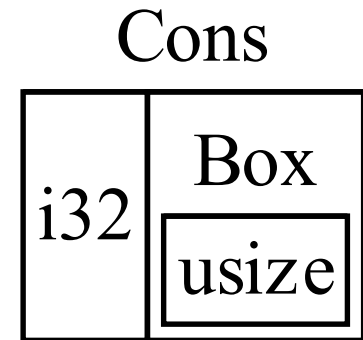
```
1 | enum List {
  | ^^^^^^^^^
2 |     Cons(i32, List),
  |               ---- recursive without indirection
```

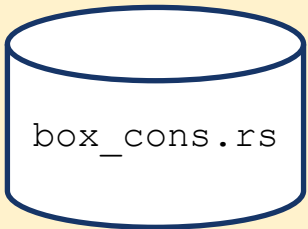
Cons



- Utilizzando il Box, il compilatore riesce a definire la dimensione della struttura dati.

```
enum List {  
    Cons(i32, Box<List>),  
    Nil,  
}
```





```
#[derive(Debug)]
enum List {
    Cons(i32, Box<List>),
    Nil,
}

fn print_list(list: &List) {
    match list {
        Cons(value, next) => {
            print!("{}", value);
            print_list(next);
        }
        Nil => {
            println!("Nil");
        }
    }
}

fn main() {
    let list = Cons(0, Box::new(Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))))));
    println!("{:?}", list);

    let a = Cons(3, Box::new(Nil));
    let b = Cons(2, Box::new(a));
    let c = Cons(1, Box::new(b));
    let head = Cons(0, Box::new(c));

    let mut current_node = &head;
    while let Cons(value, next) = current_node { // cicla finché trova la variante Cons
        println!("Value: {}", value);
        current_node = next;
    }
    print_list(&head);
}
```

```
while let Cons(value,next) = current_node {
    println!("current node : {}", value);
    current_node = next;
}

e

print_list(&head)
```

sono due modi equivalenti per scrivere la stessa cosa

Albero Binario

```
// Definizione di una struttura per un albero binario
#[derive(Debug)]
struct Node {
    value: i32,
    left: Option<Box<Node>>,
    right: Option<Box<Node>>,
}

impl Node {
    fn new(value: i32) -> Self {
        Node { value, left: None, right: None }
    }
}
```

```

struct BinarySearchTree {
    root: Option<Box<Node>>,
}

impl BinarySearchTree {
    fn new() -> Self {
        BinarySearchTree { root: None }
    }
    fn insert(&mut self, value: i32) {
        let new_node = Box::new(Node::new(value));
        match &mut self.root {
            None => self.root = Some(new_node),
            Some(root) => root.insert_node(value),
        }
    }
    fn contains(&self, value: i32) -> bool {
        let mut current = &self.root;
        while let Some(node) = current {
            match value.cmp(&node.value) {
                std::cmp::Ordering::Equal    => return true,
                std::cmp::Ordering::Less      => current = &node.left,
                std::cmp::Ordering::Greater  => current = &node.right,
            }
        }
        false
    }
}

```

```

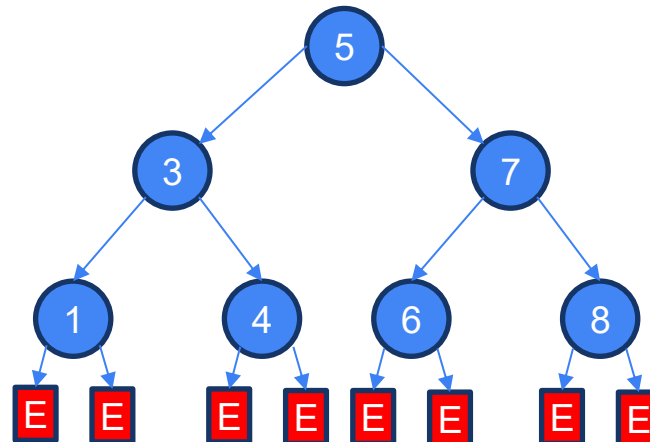
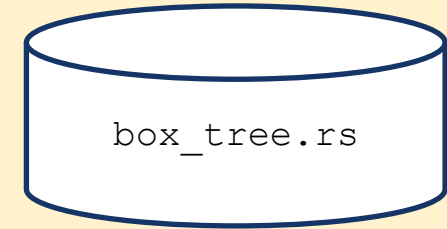
impl Node {
    fn insert_node(&mut self, value: i32) {
        if value < self.value {
            match &mut self.left {
                None => self.left = Some(Box::new(Node::new(value))),
                Some(left) => left.insert_node(value),
            }
        } else {
            match &mut self.right {
                None => self.right = Some(Box::new(Node::new(value))),
                Some(right) => right.insert_node(value),
            }
        }
    }
}

```

```
// Visita in-order

fn inorder(node: &Option<Box<Node>>) {
    if let Some(n) = node {
        inorder(&n.left);
        print!("{}", n.value);
        inorder(&n.right);
    }
}
```

```
fn main() {  
    let mut bst = BinarySearchTree::new();  
  
    for val in [5, 3, 7, 1, 4, 6, 8] {  
        bst.insert(val);  
    }  
  
    inorder(&bst.root); // 1 3 4 5 6 7 8  
  
    println!("\nContiene 4? {}", bst.contains(4)); // true  
    println!("Contiene 9? {}", bst.contains(9));  // false  
}
```



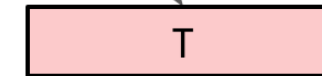
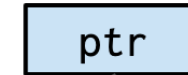
std::rc::Rc<T>

Reference Counted

- Nelle situazioni in cui occorre disporre di **più possessori** di uno stesso dato **immutabile**, è possibile usare questo smart pointer
- Internamente mantiene
 - una copia del dato
 - due contatori: il primo indica quante copie del puntatore esistono (strong), il secondo quanti riferimenti deboli (weak) sono presenti
- è possibile avere un'ulteriore copia del contatore attraverso la chiamata del metodo **clone()**
 - Ogni volta che questo **puntatore viene clonato**, il contatore **strong** viene incrementato
- Quando il puntatore esce dal proprio scope, il contatore viene decrementato: se il risultato è 0, il blocco **viene rilasciato**
- **Rc<T>** si presta a realizzare alberi e grafi aciclici
 - Può essere usato solo in contesti **single-thread**

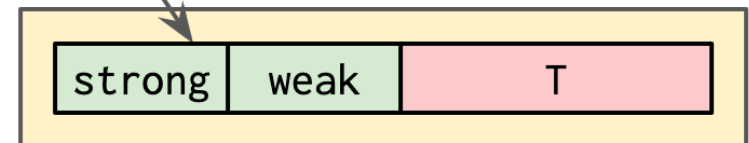
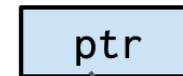
Attenzione a questa definizione

ref: &T



where T: Sized

rc: Rc<T>



ref = &*rc;

È un puntatore che permette di avere un numero di possessori superiore ad 1.

T è il dato a cui punta il puntatore Rc ad una zona di memoria che contiene alcune informazioni.

Le informazioni strong e weak sono abbinate a T e dicono quanti sono le variabili che puntano al dato.

per ora focalizziamoci solo al dato strong, che rappresenta il riferimento forte. Nel disegno della slide precedente strong vale 1. Il disegno azzurro è la variabile che punta al dato possedendolo. Noi possiamo avere più rettangoli azzurri e quindi posso avere più possessori del dato. Per ogni rettangolo azzurro che ho, strong si incrementa di 1.

Quindi, la variabile Rc creata punta ad un certo numero di dati; il valore di strong è proprio uguale al numero dei dati puntati

1h 56' 30" Il possessore possiede il dato tendenzialmente immutabile → l'Rc è fatto per strutture dati in cui ho diversi possessori di un dato che è immutabile, anche se non è proprio vero che sia immutabile.

Per creare un altro riferimento, utilizzo il metodo .clone(), ovvero creo un'altra variabile che punta allo stesso dato e questo incrementa strong di 1.

La struttura dati (o puntatore) Rc si può usare solamente in ambienti single-thread; non si può usare in contesti multi-thread perché l'incremento del campo strong non sarebbe atomico → se io ho due thread che fanno l'operazione di clone contemporaneamente, non è garantito che l'aggiornamento di strong sia corretto. Quindi questa struttura non è thread safe.

c'è un'operazione implicita di aggiornamento del contatore, e questa operazione con l'Rc non è atomica e quindi è rischioso

i metodi che operano sul puntatore Rc sono delle funzioni associate, ovvero non possono essere usate con la notazione `.nome_funzione()`; non c'è l'oggetto ricevitore cioè non c'è il `self`, in quanto si vuole dire esplicitamente che l'operazione è fatta sul puntatore e non sul dato. Devo passare come parametro un puntatore

`std::rc::Rc<T>`

- Rc dispone di metodi che operano sullo Smart Pointer
- Per evitare problemi di omonimia con i metodi contenuti nel dato incapsulato, tutti i metodi di Rc sono funzioni associate e non prevedono il parametro `self`:
 - `pub fn strong_count(this: &Rc<T>) -> usize`
 - Chiamando `this` (e non `self`) il parametro che indica l'istanza, non è possibile utilizzare la notazione puntata per invocare i metodi, ma occorre richiamarli nella forma estesa `Rc::<T>::strong_count(&a)`
- Per motivi di efficienza, l'operazione di incremento e decremento sui campi privati `strong_count` e `weak_count` non è *thread-safe*
 - Per questo motivo, non è possibile utilizzare questo smart pointer in un contesto concorrente
 - Esiste un altro smart pointer, trattato in seguito, che supera questo limite: `std::sync::Arc<T>`

Text

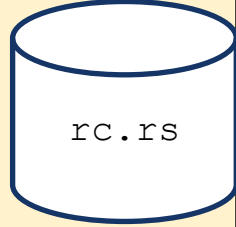
```

use std::rc::Rc;
fn main() {
    // Creiamo un valore Rc contenente una stringa.
    let rc_esempio = Rc::new("Esempio Rc".to_string());
    {
        // Cloniamo rc_esempio per creare un nuovo puntatore allo stesso valore.
        let rc_a = Rc::clone(&rc_esempio);
        println!("Conteggio referenze di rc_a: {}", Rc::strong_count(&rc_a));
        {
            // Cloniamo ancora per creare un altro puntatore.
            let rc_b = Rc::clone(&rc_a);
            println!("Conteggio referenze di rc_b: {}", Rc::strong_count(&rc_b));
            println!("Conteggio referenze di rc_a: {}", Rc::strong_count(&rc_a));

            // I due Rc sono uguali se i loro valori interni sono uguali.
            println!("rc_a e rc_b sono uguali: {}", rc_a.eq(&rc_b));

            // Possiamo usare direttamente i metodi del valore interno.
            println!("Lunghezza del valore dentro rc_a: {}", rc_a.len());
        }
        // Quando rc_b esce dallo scope, il conteggio delle referenze diminuisce.
        println!("Conteggio referenze di rc_a dopo che rc_b è uscito: {}",
Rc::strong_count(&rc_a));
    }
    println!("Conteggio referenze di rc_esempio dopo che rc_a è uscito: {}",
Rc::strong_count(&rc_esempio));
    // Quando rc_esempio esce dallo scope, rc_esempio e il valore vengono eliminati.
}

```



rc_a.len() applica
la deref coercion, cioè
Rust spacchetta il puntatore
Rc e applica il metodo
direttamente al dato contenuto

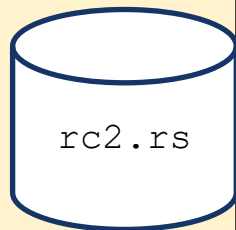
guarda il quaderno della teoria per la spiegazione dettagliata.
Quando io applico il metodo associato, e lo chiamo così perchè viene chiamato con Rc::clone ovvero con i due punti, sto di fatto creando un clone del puntatore e non sto minimamente toccando la struttura dati nello heap.

```
use std::rc::Rc;
```

2h 07' 30"

04-05-2026

```
fn main() {  
    let valore = Rc::new(5);  
  
    println!("Valore: {:?}", valore);  
  
    let copia = Rc::clone(&valore);  
    println!("Valore di copia: {:?}", copia);  
  
    let rif_rc = &*copia;  
  
    visualizza(rif_rc);  
    println!("Leggo questi valori: {} e {}", valore, copia);  
}  
  
fn visualizza (par: &i32) {  
    println!("Ho un riferimento a questo valore {}", par);  
}
```



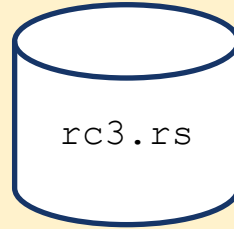
attraverso la notazione `&*` ottengo un riferimento del dato T che però deve essere Sized

guarda il quaderno degli
appunti

2h 09' 50"

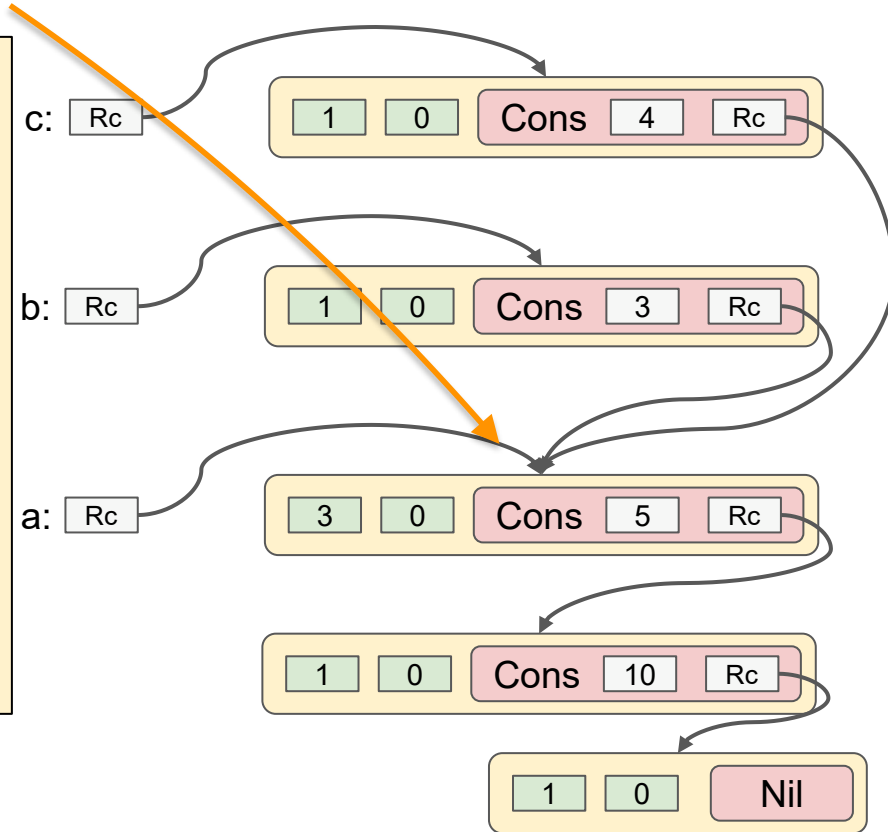
la struttura allocata per il b contiene il valore 3 e un Rc che punta alla struttura dati dello heap puntata
dalla variabile Rc "a"

```
enum List {  
    Cons(i32, Rc<List>),  
    Nil,  
}
```



```
fn main() {  
    let a = Rc::new(  
        Cons(5, Rc::new(Cons(10, Rc::new(Nil))));  
  
    let b = Rc::new(Cons(3, Rc::clone(&a)));  
    let c = Rc::new(Cons(4, Rc::clone(&a)));  
}
```

Text



Mutabilità del dato puntato da Rc

- Il dato puntato da Rc è mutabile solo se non esistono altri riferimenti al dato (per la regola del borrow checker)
- Il metodo da utilizzare è `get_mut(this: &Rc<T>) -> Option<& mut T>`
 - Se non esistono altri puntatori (strong o weak) allo stesso dato, `get_mut` restituirà `Some(& mut T)` permettendo di modificare il valore contenuto
 - Altrimenti restituirà `None`, poiché non è possibile avere più riferimenti mutabili allo stesso valore.

```
use std::rc::Rc;
```

con il metodo associato `get_mut` posso modificare il valore contenuto nella variabile di tipo `Rc`. Per poter fare questo occorre chiamare l'istruzione precedendola dal `match`, perchè non sempre è possibile ottenere un riferimento mutabile del dato contenuto nell'`Rc` che voglio modificare

```
fn main() {
```

```
    let mut valore = Rc::new(5);
```

```
    {
```

```
        println!("Valore: {:?}", valore);
```

```
        let copia = Rc::clone(&valore);
```

```
        println!("Valore di copia: {:?}", copia);
```

```
        match Rc::get_mut(&mut valore) {
```

```
            Some(v) => *v += 10, // Modifica il valore se è possibile.
```

```
            None => println!("Non è possibile ottenere un riferimento mutabile."),
```

```
        }
```

```
    }
```

```
    // Prova a ottenere un riferimento mutabile.
```

```
    match Rc::get_mut(&mut valore) {
```

```
        Some(v) => *v += 10, // Modifica il valore se è possibile.
```

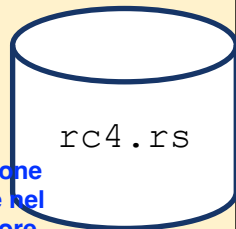
```
        None => println!("Non è possibile ottenere un riferimento mutabile."),
```

```
    }
```

```
    println!("Valore: {:?}", valore); // Stampa 15
```

```
}
```

Infatti, poichè in questo caso esiste la variabile `let copia`, che è un altro possessore del dato, l'esecuzione dell'istruzione `match Rc::get_mut(&mut valore)` finisce nel ramo negativo dato da `None`. Praticamente se il contatore `strong` è maggiore di 1 non posso modificare il dato indirizzato dall'`Rc`



in questa istruzione ritento di modificare il dato e posso farlo per il contatore `Strong` vale 1. Se noti, il secondo possessore, ovvero `let copia`, è morto perchè è finito il suo scope

```
use std::rc::Rc;
#[derive(Debug)]
struct Node {
    value: i32,
    children: Vec<Rc<Node>>,
}
```

```
fn main() {
    let mut nipote1 = Rc::new(Node {
        value: 3,
        children: vec![],
    });
```

```
    let nipote2 = Rc::new(Node {
        value: 6,
        children: vec![],
    });
```

```
    let padre = Rc::new(Node {
        value: 9,
        children: vec![Rc::clone(&nipote1), Rc::clone(&nipote2)],
    });
```

```
    let nonno = Rc::new(Node {
        value: 27,
        children: vec![Rc::clone(&padre)],
    });
```

```
    match Rc::get_mut(&mut nipote1) {
        Some(v) => v.children.push(Rc::clone(&nonno)),
        None => println!("Non è possibile ottenere un riferimento mutabile."),
    }
```

```
    println!("{:?}", nonno);
}
```

tuttavia il match in realtà va nel ramo None perchè la variabile padre di tipo Rc possiede un clone di nipote1, la stessa variabile Rc il cui dato stiamo tentando di modificarlo, ma questo non è possibile



la v dentro Some(v) rappresenta la Struct Node specifica della variabile nipote1.

Come si comporta e perché?

qui cerco di modificare il vettore tentando di pushare un clone della variabile Rc nonno

std::rc::Weak<T>

2h 19' 30"

- Se si costruisse, usando **Rc<T>**, una sequenza circolare di puntatori, la memoria allocata non potrebbe più essere rilasciata
 - la catena dei puntatori terrebbe in vita tutti i blocchi, garantendo che il conteggio dei riferimenti valga almeno 1
- E' possibile creare una struttura con dipendenze circolari utilizzando il tipo **Weak<T>**
 - Esso è una versione di **Rc** che contiene un riferimento senza possesso al blocco allocato
 - se ciò a cui punto è ancora vivo **ci arrivo**
 - se ciò a cui punto **non è più vivo, non ci arrivo**
 - dunque **non partecipo al processo di tenuta in vita**, ma **se l'oggetto è vivo ci posso arrivare**.
- Si crea un valore di tipo **Weak<T>** a partire da un valore di tipo **Rc<T>** con il **metodo Rc::downgrade(&rc)**
 - Se il valore originale è ancora in vita (ovvero se **strong_counter() > 0**), è possibile costruire un nuovo valore di tipo **Rc<T>** invocando il metodo **upgrade()**
 - Esso **ritorna un valore di tipo Option<Rc<T>>**



- Un puntatore Weak può essere generato solo a partire da un Rc, attraverso il metodo `downgrade`
- Un Weak non è direttamente dereferenzabile (non gode del tratto Deref). Se faccio `*w` di un oggetto Weak, non ottengo l'accesso al campo (errore di compilazione).
- Per poter accedere al dato a cui quel Weak punta devo fare upgrade
 - Se il puntatore Rc è ancora vivo, il Weak si tramuta in un Rc, in modo da poterlo possedere, almeno temporaneamente.
 - Al termine dell'elaborazione, scompare il riferimento strong ottenuto, ma rimane la variabile di tipo Weak che può eventualmente essere riutilizzata con un successivo upgrade.

il weak rappresenta un riferimento debole. Vediamo la differenza tra un riferimento forte e un riferimento debole:

il riferimento forte definisce la durata in vita di un dato; quindi il dato, ovvero la struttura dati rappresentata in giallo alla slide 38, viene rilasciata quando il contatore va a zero. Il contatore di Strong si decrementa quando la variabile Rc esce dal suo scope. QUando l'ultimo Rc esce dallo scope, il contatore di strong va a zero e la struttura dati in giallo viene rilasciata

il riferimento debole, nel contesto di un grafo ciclico esiste sempre un riferimento al dato e quindi non riuscirei mai a deallocare la struttura dati. Se ho un grafo in cui c'è un anello in cui due nodi, ad esempio, reciprocamente puntano l'uno all'altro non ci sarebbe mai una situazione in cui il contatore dei riferimenti vada a zero. per avere una condizione di uscita stabile ci vuole un riferimento debole, che non influisce sulla permanenza del dato in memoria ma che permette però l'accesso al dato.

L'ACCESSO AL DATO ATTRAVERSO UN PUNTATORE DEBOLE AVVIENE SOLO SE IL DATO ESISTE, perchè potrebbe essere che il contatore strong vale 0, ma esiste un riferimento weak che tenta di accedere al dato, ma questo non esiste più.

Posso generare una variabile Weak a partire da una variabile di tipo Rc con il metodo associato Rc::downgrade(&rc)

Per poter accedere al dato devo passare dal tipo weak al tipo Rc e questo è possibile tramite la funzione upgrade, che ritorna un Option, perchè devo sempre verificare che il dato sia in vita.

Il puntatore weak non può essere deferenziato tramite l'asterisco per l'accesso al dato. Per poter avere l'accesso al dato tramite il Weak devo utilizzare il metodo .upgrade(). Ogni volta che faccio l'upgrade di una variabile Weak mi viene restituito Option<Rc<T>>. Pertanto, per estrarre il valore dell'Rc devo utilizzare il match e in particolare il match consuma la variabile da cui ho estratto il valore

```

use std::rc::Rc;
fn main() {
    let five = Rc::new(5);
    let _ten = Rc::clone(&five);

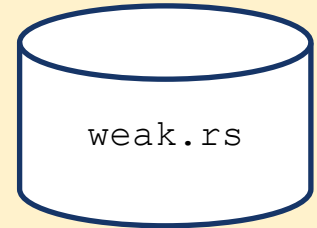
    let weak_five = Rc::downgrade(&five);

    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five)); // 2
    println!("Conteggio referenze weak di five: {}", Rc::weak_count(&five)); // 1

    let strong_five = weak_five.upgrade(); // strong_five è un Option<Rc<T>>

    println!("{:?}", strong_five); // Some(5)
    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five)); // 3
    match strong_five {
        Some(rc_strong_five) => println!("{:?}", rc_strong_five),
        None => println!("Il dato non è disponibile "),
    }
    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five));
}

```



qui consumo la variabile `strong_five` perchè sto estraendo il dato contenuto nella variabile di tipo `Option<Rc<T>>` tramite il `match`. Pertanto, dopo il `match` la variabile viene consumata e quindi il contatore `strong` viene decrementato di 1. Questo è il motivo per cui alla fine la `print` stampa 2

Che valore viene stampato?
Perché?

Risposta

- La sequenza del numero di contatori forti è:
 - $2 \Rightarrow 3 \Rightarrow 2$
- `upgrade()` aumenta il conteggio strong perché crea un nuovo Rc
- alla fine del match, quel nuovo Rc viene distrutto, quindi il conteggio torna a 2.

```

use std::rc::Rc;
fn main() {
    let five = Rc::new(5);
    let _ten = Rc::clone(&five);

    let weak_five = Rc::downgrade(&five);

    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five));
    println!("Conteggio referenze weak di five: {}", Rc::weak_count(&five));

    let strong_five= weak_five.upgrade(); // strong_five è un Option<Rc<T>>

    println!("{:?}", strong_five);
    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five));
    match &strong_five {
        Some(rc_strong_five) => println!("{:?}", rc_strong_five),
        None => println!("Il dato non è disponibile "),
    }
    println!("Conteggio referenze strong di five: {}", Rc::strong_count(&five));
}

```

2h 31'00" 04-05-2026



in questo esempio, simile a quello della slide 52, il contatore forte della variabile five di tipo Rc vale tre perchè il match opera su un riferimento che non distrugge la variabile Rc strong_five

Che valore viene stampato?
Perché?

```
use std::rc::Rc;
fn main() {
    let five = Rc::new(5);
```

esempio importante



```
    let weak_five = Rc::downgrade(&five);
    println!("{:?}", five);
    drop(five);

    let strong_five = weak_five.upgrade();
    match strong_five {
        Some(rc_strong_five) => println!("{:?}", rc_strong_five),
        None => println!("Il dato non esiste più"),
    }
}
```

poichè ho fatto il drop della variabile Rc, se io ricreo un Rc tramite .upgrade() che ricordiamo lo incapsula in un Option, alla fine l'Option contiene None perchè il possessore originale è stato ucciso con il drop. Tuttavia il dato nello heap rimane ancora allocato, altrimenti l'upgrade non potrebbe funzionare, nel senso che non si saprebbe su cosa viene applicato il metodo. Nel senso che funziona, ma il valore di ritorno è None. La struttura dati nello heap viene deallocata solo quando si esce dallo scope.

REGOLA IMPORTANTE: LA STRUTTURA DATI DELL'RC NON È PIU' ACCESSIBILE QUANDO L'ULTIMO PUNTATORE FORTE MUORE. ESISTE IN QUANTO UN RIFERIMENTO DEBOLE PUO' CHIEDERE L'ACCESSO ALLA STRUTTURA DATI CHE VIENE NEGATO. ESISTE MA NON È ACCESSIBILE

std::cell

lezione del 06-05-2026

- Il *borrow checker* garantisce, in fase di compilazione, che dato un valore di tipo T in ogni momento valgano i seguenti invarianti, mutuamente esclusivi
 - Non esista alcun riferimento al valore al di là del suo possessore
 - Esistano uno o più riferimenti immutabili (&T) - aliasing
 - Esista un solo riferimento mutabile (&mut T) - mutabilità
- Esistono situazioni in cui l'analisi statica eseguita in fase compilazione è troppo restrittiva
 - Il modulo std::cell offre alcuni contenitori che consentono una mutabilità condivisa e controllata
 - I tipi offerti possono funzionare solo in contesti non concorrenti (basati su singolo thread)

le primitive di sincronizzazione garantiscono che l'accesso ad un dato possono avvenire in modo protetto. Attorno a questi tipi che stiamo vedendo, tra cui cell, non c'è alcun meccanismo di sincronizzazione, e pertanto non possono essere usati in contesti multithread

Guardiamo la rappresentazione del dato di tipo Cell. Il dato è memorizzato nello Stack, per cui non fornisce strumenti di propagazione del dato dopo la fine della funzione.
Con questo tipo inserisco delle caratteristiche gestite esclusivamente dal compilatore, che gestisce la variabile di tipo Cell in modo singolare. La proprietà del Cell è che garantisce mutabilità interna.
Il tipo Cell non permette di creare riferimenti al dato, e permette solo fare operazioni di lettura(swap) o scrittura

std::cell::Cell<T>



- La struct `std::cell::Cell<T>` implementa la mutabilità del dato contenuto al suo interno (**memorizzato nello stack**) attraverso metodi che non richiedono la mutabilità del contenitore
- `Cell<T>` non introduce nessuna meta-informazione aggiuntiva !
- Si dice che Cell implementa un meccanismo di *interior mutability* (nell'ambito di una *exterior immutability*): un dato conosciuto con una reference può essere modificato (facendo una operazione di swap)
- Le Cell permettono di bypassare le regole di analisi statica fatta dal borrow checker del compilatore:
 - in un programma single thread, si può creare una variabile di tipo Cell, conosciuta come reference semplice, ma sostituire il dato X con il dato Y
 - `Cell<T>` non consente di creare riferimenti al dato contenuto al suo interno, ma solo alla stessa Cell<T>.

std::cell::Cell<T>

- Una istanza di **Cell<T>** viene creata con il metodo **new(value: T) -> Cell<T>**
- Il metodo **set(&self, val: T)** imposta il valore contenuto nella cella con il valore passato come parametro
- Il metodo **get(&self) -> T** che restituisce il dato contenuto al suo interno
 - a condizione che T implementi il tratto Copy
- Il metodo **take(&self) -> T** restituisce il valore contenuto, sostituendolo con il risultato dell'invocazione di **Default::default()**
 - a condizione che T implementi il tratto Default
- Il metodo **replace(&self, val:T) -> T** sostituisce il valore contenuto nella cella con quello passato come parametro e lo restituisce come risultato
 - questo metodo non pone restrizioni sul tipo di dato
- Il metodo **into_inner(&self) -> T** consuma la cella e restituisce il valore contenuto
 - anche in questo caso, può essere usato con ogni tipo di dato

```
use std::cell::Cell;
#[derive(Debug)]
struct SomeStruct {
    a: u8,          // il campo a è soggetto a tutti i vincoli del borrow checker
    b: Cell<u8>,    // il campo b è soggetto ad un rilassamento di questi vincoli
}
fn main() {
```

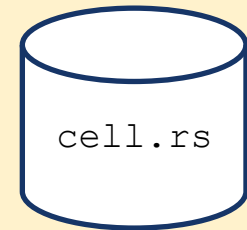
7'30"

```
    let my_struct = SomeStruct {
        a: 0,
        b: Cell::new(1),
    };
```

```
    // my_struct.a = 100;
    // ERRORE: `my_struct` è immutabile
```

```
    let copy_struct = &my_struct;
    my_struct.b.set(100);
```

nonostante io abbia creato un riferimento immutabile della variabile di tipo SomeStruct, io posso modificare il campo b di tipo Cell. Un'operazione di questo tipo sarebbe illegale dal punto di vista del borrow checker.



Quindi vediamo che il tipo Cell ci permette di bypassare alcuni vincoli del borrow checker, come modificare il campo di una variabile quando esiste un riferimento immutabile in vita.

```
    // OK: anche se esiste un Reference, `b` è una Cell e può essere modificata

    println!("{:?}", my_struct);
    println!("{:?}", copy_struct);
}
```

```
use std::cell::Cell;
struct Contatore {
    conteggio: Cell<u32>,
}
impl Contatore {
    fn new() -> Contatore {
        Contatore {
            conteggio: Cell::new(0),
        }
    }
    fn incrementa(&self) {
        let conteggio_attuale = self.conteggio.get() ;
        self.conteggio.set(conteggio_attuale + 1);
    }
    fn decrementa(&self) {
        let conteggio_attuale = self.conteggio.get();
        self.conteggio.set(conteggio_attuale - 1);
    }
    fn leggi(&self) -> u32 {
        self.conteggio.get()
    }
}
fn main() {
    let contatore = Contatore::new();
    contatore.incrementa();
    contatore.incrementa();
    println!("Conteggio: {}", contatore.leggi());
    contatore.decrementa();
    println!("Conteggio: {}", contatore.leggi());
}
```



```
use std::cell::Cell;

fn main() {
    // Creiamo una Cell contenente un valore iniziale.
    let cell = Cell::new(42);

    // Utilizziamo il metodo take per estrarre il valore dalla Cell.
    let taken_value = cell.take();
    println!("Valore estratto dalla Cell: {}", taken_value); // 42

    // Utilizziamo il metodo replace per sostituire il valore all'interno della Cell
    // e ottenere quello precedente.

    let previous_value = cell.replace(99);
    println!("Valore precedente nella Cell: {}", previous_value); // 0

    // Utilizziamo il metodo into_inner per estrarre il valore contenuto nella Cell.
    let inner_value = cell.into_inner();
    println!("Valore estratto dalla Cell con into_inner: {}", inner_value); // 99

    // la cella è stata mossa e dunque non è più accessibile
    // let taken_value = cell.take();
}
```



```

enum List {
    Cons(Rc<Cell<i32>>, Rc<List>),
    Nil,
}
fn print_list(list: &List) {
    let mut current = list;
    print!("{}", "");
    while let Cons(val, next) = current {
        print!("{}", val.get());
        current = next;
        if let Cons(_, _) = current {
            print!("{}", "");
        }
        else { print!("{}", Nil); }
    }
    println!("{}", "");
}
fn main() {
    let a = Rc::new(Cons(Rc::new(Cell::new(5)), Rc::new(Nil)));
    let b = Rc::new(Cons(Rc::new(Cell::new(6)), Rc::clone(&a)));
    let c = Rc::new(Cons(Rc::new(Cell::new(100)), Rc::clone(&a)));
    let head = Cons(Rc::new(Cell::new(10)), Rc::clone(&b));
    if let Cons(val, _) = &*a { // Modifichiamo il valore interno di `a`
        val.set(42);
        println!("Valore aggiornato in `a`: {}", val.get());
    }
    print_list(&head); // [10, 6, 42, Nil]
    print_list(&c);    // [100, 42, Nil]
}

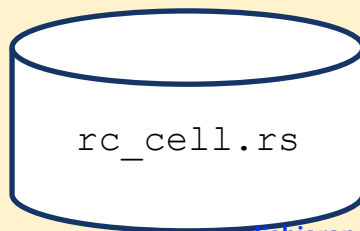
```

10'10"

codice da vedere meglio

Combinazione di Cell e Rc

Un Rc<T> che tiene un Cell<T> corrisponde ad un dato che può avere molti proprietari e che può essere internamente mutato.



dichiarando la variabile di tipo Cell è possibile aggiornare il dato senza problemi. Anche se ho più proprietari, io posso aggiornare il dato proprio perchè è di tipo cell. L'uso di questo smart pointer permette di bypassare alcuni vincoli.

std::cell::RefCell<T>



- **Cell<T>** non consente di creare riferimenti al dato contenuto al suo interno, ma solo di inserire, estrarre o sostituire il valore
- La struct **std::cell::RefCell<T>** permette di creare un dato con **un unico possessore**, a cui è possibile accedere attraverso smart pointer che simulano il comportamento di riferimenti condivisi e mutabili
 - Ma la cui compatibilità con le regole del borrow checker è stabilita in fase di esecuzione e non di compilazione
 - Eventuali tentativi di violazione delle regole (in fase di run time) generano una condizione di panic, comportando la terminazione del thread corrente
- Oltre a incapsulare il dato, ha un campo che dice se il dato è stato imprestato
 - Se non ce l'ha nessuno, posso accedere al dato
 - Se ce l'ha qualcuno in lettura e
 - Viene chiesto in lettura, si segna che sono in n+1 ad avercelo in lettura
 - Viene chiesto in scrittura, da un panic
 - Se ce l'ha qualcuno in scrittura, e viene chiesto in lettura o scrittura, da un panic.

La struttura RefCell aggiunge al nostro dato un'informazione relativa ai possibili prestiti: conta i possibili prestiti e valuta se il prestito è mutabile o immutabile. L'operazione di verifica del borrow checker per questo tipo di variabile non viene fatto a runtime, ma piuttosto in tempo di compialzione. Se c'è almeno un prestito in lettura, il programma ca in stato di panico. Dlversamente dalle re

std::cell::RefCell<T>

- **borrow(&self) -> Ref<'_, T>** restituisce uno smart pointer che implementa il tratto **Deref<T>**
 - Oppure provoca un panic se è già presente un riferimento mutabile
- **borrow_mut(&self) -> RefMut<'_, T>** restituisce uno smart pointer che implementa il tratto **DerefMut<T>**
 - Oppure provoca un panic se è già presente un riferimento semplice
- **try_borrow(&self) -> Result<Ref<T>, BorrowError>**: restituisce un Result contenente un riferimento immutabile al valore, se possibile, senza panicare se non è possibile ottenerlo
- **try_borrow_mut(&self) -> Result<RefMut<T>, BorrowMutError>**: restituisce un Result contenente un riferimento mutabile al valore, se possibile, senza panicare se non è possibile ottenerlo
- **into_inner(self) -> T**: consuma la RefCell e restituisce il valore contenuto al suo interno (può essere eseguito solo se non esistono riferimenti attivi)
- **get_mut(&mut self) -> &mut T**: restituisce un riferimento mutabile al valore contenuto nella RefCell se non ci sono altri riferimenti attivi (**controllato dal borrow checker al tempo di compilazione**)

Trova le differenze

```
fn main() {  
    use std::cell::RefCell;  
    let mut cell = RefCell::new(42);  
    let m = cell.borrow();  
    println!("The value is: {}", m);  
    let value = cell.get_mut();  
    *value += 1;  
}
```



Text



```
fn main() {  
    let c = RefCell::new(5);  
    let _r1 = c.borrow();  
    let mut r2 = c.borrow_mut();  
    *r2 += 1;  
}
```



thread 'main' panicked at src\main.rs:5:17:
already borrowed: BorrowMutError

```

use std::cell::RefCell;
fn print_refcell(rc: &RefCell<i32>) {
    println!("Il dato è: {:?}" , *rc.borrow());
}
fn main() {
    let mut c = RefCell::new(5);
    print_refcell(&c); // 5
    {
        *c.get_mut() += 5;

        print_refcell(&c); // 10

        let mut m = c.borrow_mut();

        if c.try_borrow().is_err()
        {
            println!("Non posso fare un altro prestito"); // Si entra qua
        }
        *m = 6;
        drop(m); // operazione necessaria per poter avere un riferimento a 'c'
        print_refcell(&c); // 6
    }
    if c.try_borrow().is_ok()
    {
        println!("Posso fare un altro prestito");
        let m = c.borrow();
        print_refcell(&c); // 6
        if c.try_borrow_mut().is_ok()
        {
            println!("Posso fare un altro prestito"); // !!! non si entra qua !!!
            let mut m2 = c.borrow_mut();
            *m2 = 8;
        }
        drop(m); // necessario per permettere il successivo into_inner
        let val = c.into_inner();
        println!("Valore Finale dopo la distruzione del RefCell: {:?}" , val);
    }
}

```

20'00"

refcell.rs

```
use std::cell::RefCell;
```

```
fn main() {  
    let numero = RefCell::new(5);  
  
    // Tentiamo di ottenere un riferimento mutabile alla RefCell  
    match numero.try_borrow_mut() {  
        Ok(mut riferimento_mutabile) => {  
            println!("Riferimento mutabile ottenuto con successo.");  
            // Ora possiamo modificare il valore all'interno della RefCell  
            *riferimento_mutabile = 10;  
            // il riferimento mutabile ottenuto non è stato mosso in una variabile con un  
            // binding per assegnare il valore ad una variabile memorizzata  
        }  
        Err(_) => {  
            println!("Impossibile ottenere un riferimento mutabile perché è già in uso.");  
        }  
    }  
    // numero di riferimento in essere: 0  
    // prendiamo un riferimento in lettura e leggiamo il valore  
    println!("Il valore all'interno della RefCell è ora: {}", *numero.borrow());  
    // anche qua non c'è binding, numero di riferimento in essere: 0  
  
    // Tentiamo di ottenere un altro riferimento mutabile  
    match numero.try_borrow_mut() {  
        Ok(mut riferimento_mutabile) => {  
            println!("Si può fare!");  
            *riferimento_mutabile = 100;  
        }  
        Err(_) => {  
            println!("Impossibile ottenere un secondo riferimento mutabile.");  
        }  
    }  
    println!("Il valore finale della RefCell è: {:?}", *numero.borrow());  
}
```



refcell3.rs

```
use std::cell::RefCell;
fn main() {
    let c = RefCell::new(5);
    println!("Il dato è {:?}", c.borrow()); // 5

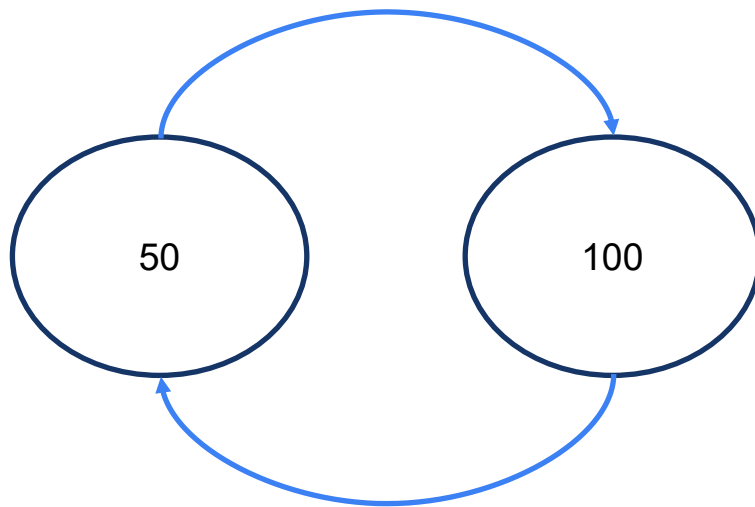
    if c.try_borrow_mut().is_ok()
    {
        println!("Posso fare un prestito mutabile e lo faccio");
        let mut m2 = c.borrow_mut();
        *m2 = 8;
    }
    println!("Il dato è {:?}", c.borrow()); // 8
    match c.try_borrow_mut() {
        Ok(mut riferimento_mutabile) => {
            *riferimento_mutabile = 100;
        }
        Err(_) => {
            println!("Impossibile ottenere un secondo riferimento mutabile.");
        }
    }
    println!("Il dato è {:?}", c.borrow()); // 100
}
```



refcell4.rs

Esercizio

- Realizzare una struttura dati di questo tipo



```
struct Nodo {
    valore: i32,
    vicino: RefCell<Weak<Nodo>>,
}
```

la struct `Nodo` la costruisco con un metodo associato che restituisce un `Rc` che contiene il nodo stesso

```
impl Nodo {
    fn nuovo(valore: i32) -> Rc<Nodo> {
        Rc::new(Nodo {
            valore,
            vicino: RefCell::new(Weak::new()),
        })
    }
}
```

poichè nel grafo è previsto che i nodi siano linkati tra loro utilizzo una variabile di tipo `RefCell` per indicare il nodo vicino

```
fn imposta_vicino(nodo: &Rc<Nodo>, vicino: &Rc<Nodo>) {
    *nodo.vicino.borrow_mut() = Rc::downgrade(vicino);
}
```

```
fn ottieni_vicino(&self) -> Option<Rc<Nodo>> {
    self.vicino.borrow().upgrade()
}
```

prendo un riferimento (`.borrow()`) per effettuare l'upgrade, non voglio alterare l'attributo originale del `Nodo`. La funzione restituisce un `Option<Rc>>`

```
fn ottieni_vicino_ver2(&self) -> Option<i32> {
    match self.vicino.borrow().upgrade() {
        Some(nodo) => Some(nodo.valore),
        None => None,
    }
}
```

nel caso in cui il nodo non abbia un vicino posso restituire correttamente `None`



definisco il tipo del vicino con un riferimento debole.

poichè qui abbiamo l'esigenza di creare una struttura dati, dobbiamo necessariamente creare un tipo weak per l'attributo vicino che inizialmente è vuota

imposta_vicino è una funzione associata perchè manca &self. Viene chiamato .bottom_mut() perchè voglio modificare il valore di RefSel. Il dato contenuto è uno Weak, che viene impostato come un downgrade di Rc, e quindi sto scrivendo il puntatore al nodo vicino. Il contenuto del Weak è il riferimento all'altro nodo.

```
fn main() {  
    // Crea i due nodi  
    let nodo_a = Nodo::nuovo(50);  
    let nodo_b = Nodo::nuovo(100);  
  
    // Inizialmente i vicini sono Weak::new(), quindi ottieni_vicino() restituirà None  
    println!("Vicino di nodo_a (iniziale): {:?}", nodo_a.ottieni_vicino().map(|n| n.valore));  
    println!("Vicino di nodo_b (iniziale): {:?}", nodo_b.ottieni_vicino_ver2());  
  
    // Imposta un arco (nodo_a punta a nodo_b)  
    Nodo::imposta_vicino(&nodo_a, &nodo_b);  
    // Imposta un arco reciproco (nodo_b punta a nodo_a)  
    Nodo::imposta_vicino(&nodo_b, &nodo_a);  
  
    // Verifica i vicini dopo aver creato l'arco reciproco  
    println!("Vicino di nodo_a: {:?}", nodo_a.ottieni_vicino().unwrap().valore);  
    println!("Vicino di nodo_b: {:?}", nodo_b.ottieni_vicino().unwrap().valore);  
}
```

sono arrivato a 36'20"

1. Strutture dati a Grafo (Reticoli)

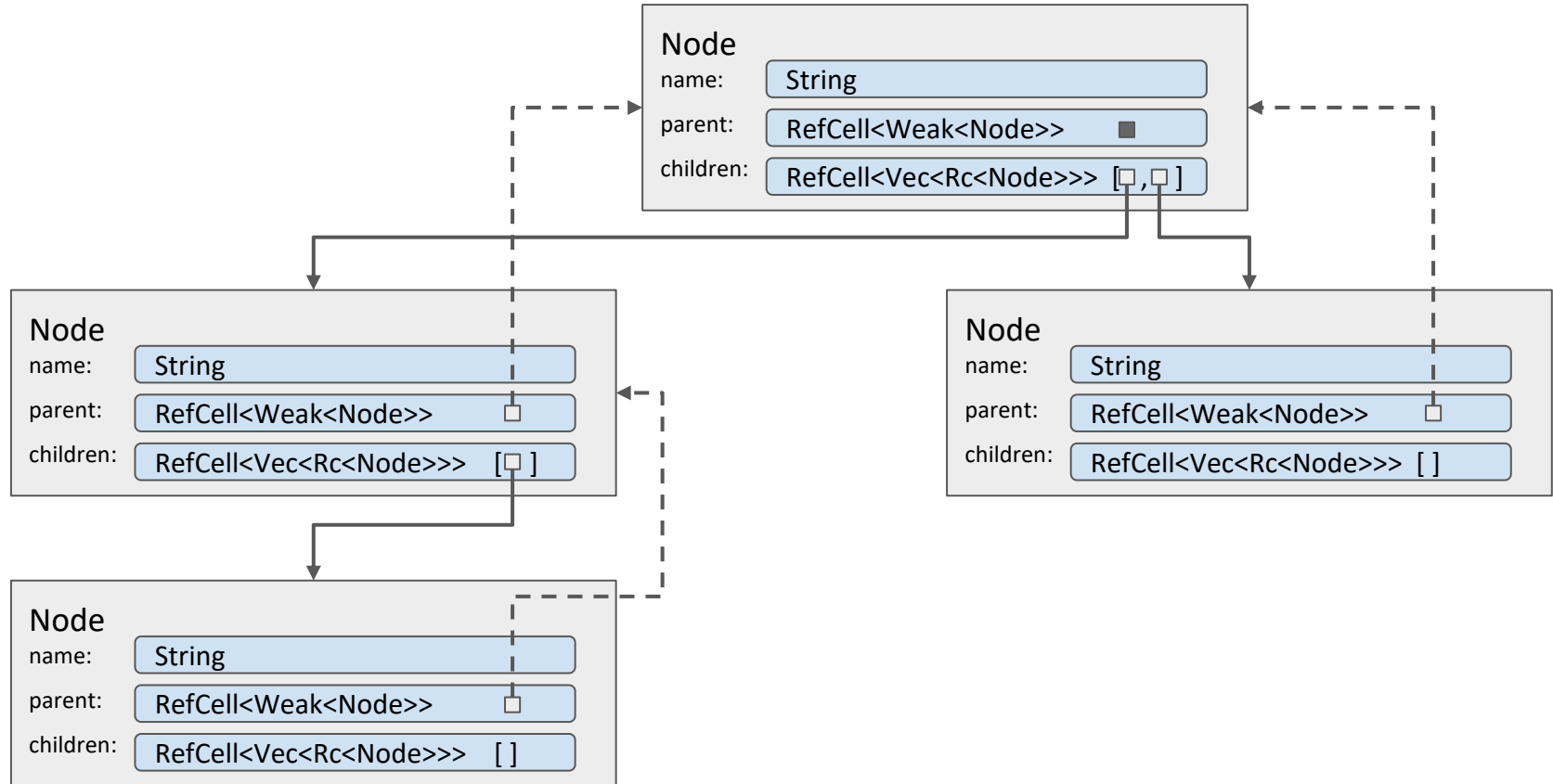
Pensa a una mappa stradale per un navigatore satellitare o a una rete di computer:

Ogni città (o router) è un Nodo.

Se la città di Torino ha una strada che va verso Milano e una che va verso Piacenza, sia il nodo di Milano che il nodo di Piacenza devono avere nel loro elenco di vicini un puntatore che punta esattamente allo stesso identico nodo di Torino.

Senza Rc, saresti costretto a duplicare il nodo di Torino in memoria per ogni collegamento, creando incongruenze (se aggiorni i dati di Torino in un punto, gli altri nodi vedrebbero ancora i dati vecchi).

Esempio: albero doppiamente collegato



Esempio: albero doppiamente collegato

```
#[derive(Debug)]
struct Node {
    name: String,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}
impl Node {
    fn new(name: &str) -> Rc<Self> {
        Rc::new(Node {
            name: name.to_string(),
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(Vec::new()),
        })
    }

    fn add_child(parent: &Rc<Node>, child: Rc<Node> ) {
        *child.parent.borrow_mut() = Rc::downgrade(&parent);
        parent.children.borrow_mut().push(child);
    }
}
```

codice da guardare

Esempio: albero doppiamente collegato

```
// ...continua

fn print_tree(node: &Rc<Node>, depth: usize) {
    println!("{}", " ".repeat(depth * 2), node.name);
    for child in node.children.borrow().iter() {
        Node::print_tree(child, depth + 1);
    }
}

// funzione per vedere uno scorrimento tramite un Weak
fn get_root(node: &Rc<Node>)-> Rc<Node> {
    let mut n = node.clone();
    loop {
        let parent = n.parent.borrow().upgrade();
        match parent {
            Some(p) => n = p.clone(),
            None => break,
        }
    }
    n
}

}
```

codice da guardare

Esempio: albero doppiamente collegato

albero.rs

```
// ...continua
```

codice da guardare

```
fn main() {  
    let root = Node::new("root");  
    let child1 = Node::new("child1");  
    let child2 = Node::new("child2");  
    Node::add_child(&root, child1.clone()); // il secondo parametro è passato con movimento  
    Node::add_child(&root, child2);         // non userò più child2 e dunque posso consumarlo  
                                            // child1 lo dovrò ancora usare e dunque passo un clone  
  
    Node::add_child(&child1, Node::new("subchild")); // qua passo direttamente l'Rc creato  
  
    Node::print_tree(&Node::get_root(&child1), 0);  
}
```

```
root  
  child1  
    subchild  
  child2
```



std::borrow::Cow<'a, B>

- Smart pointer che implementa il meccanismo **clone on write** progettato per rappresentare in modo efficiente **dati che possono essere sia presi in prestito (*borrowed*) sia posseduti (*owned*)**
- L'obiettivo primario di Cow è l'ottimizzazione delle prestazioni: **permette di lavorare con riferimenti finché è possibile** (evitando allocazioni costose sulla memoria heap) e di **effettuare una copia solo nel momento esatto in cui si rende necessaria una modifica ai dati.**
- Se si cerca di modificare il dato contenuto:
 - Se questo è in prestito, il dato viene clonato: **si prende possesso della copia e si effettua la modifica, lasciando l'originale invariato**
 - Se era già posseduto, non avviene nessuna clonazione e si opera la modifica direttamente
- Implementato sotto forma di enumerazione
 - ```
pub enum Cow<'a, B>
where B: 'a + ToOwned + ?Sized,
{
 Borrowed(&'a B), // Questo indica che il valore è preso in prestito
 // e non può essere modificato
 Owned(<B as ToOwned>::Owned), // Questo indica che il valore è di proprietà
 // e può essere modificato.
}
```



Seguendo il concetto di Copy on write, che dice “fintanto che io leggo non faccio una copia, ma se ho bisogno di scrivere ne faccio una copia”, concetto che si applica in memoria nel sistema operativo dove se ci sono dei dati che sono richiamati da più processi, le pagine possono essere mantenute condivise, ma qualora un processo debba scrivere su una pagina ne fa una copia.

In Rust, lo stesso concetto viene chiamato Clone on write: si distinguono quindi determinati tipi che sono in prestito o posseduti e si valuta se è necessario fare un’operazione di aggiornamento di un dato che è ottenuto in prestito, e se si vuole fare un aggiornamento si fa una copia.

Alcune strutture dati si prestano per questi tipi di operazione, come le &str, le slice, i vec.

una variabile incapsulata dentro un Cow può essere di due forme, perchè Cow è un enum che descrive il tipo come se è in prestito oppure posseduto. Il tipo Cow è un’attributo che si assegna al dato e lo caratterizza come dato posseduto o in prestito. Se il dato è posseduto e devo fare un’operazione di aggiornamento del dato, posso farlo tranquillamente perchè il dato è posseduto; se il dato è in prestito e non devo fare alcun aggiornamento, non si fa nessuna azione sul dato. Se invece è in prestito ma devo modificarlo, devo fare un clone del dato assumendo il possesso del nuovo dato.

```
use std::borrow::Cow;
```

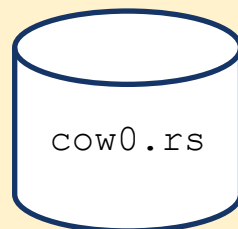
```
fn make_uppercase(input: &str) -> Cow<str> {
 if input.chars().any(|c| c.is_lowercase()) {
 Cow::Owned(input.to_uppercase())
 } else {
 Cow::Borrowed(input)
 }
}
```

il risultato è un dato che possiede la variabile: l'<str> che viene prodotto in uscita è posseduta, perchè l'operazione to\_uppercase() può essere fatto solo con le stringhe e quindi il tipo di risultato deve essere di tipo Owned. Per poter possedere il dato si è dovuto creare una copia.

```
fn main() {
 let string1 = String::from("Ciao");
 let string2 = String::from("CIAO");

 println!("{}", make_uppercase(&string1));
 println!("{}", make_uppercase(&string2));
}
```

se non ho nessuna lettere minuscola, il codice non deve effettuare nessun cambiamento: prendo l'input che avevo e lo restituisco uguale —> in particolare posso restituirlo come riferimento incapsulandolo nella variante Borrowed. Non ho quindi l'esigenza di creare una nuova stringa su cui copiare il valore in input.



Se è richiesto effettuare una modifica sul dato preso in input, allora l'idea è applicare il paradigma Copy on write —> Quindi incapsulo il dato che devo modificare nel tipo Cow, specificando la clonazione, IL CHE EQUIVALE AL POSSESSO DI UN NUOVO DATO —> Owned

Sì, la variante Cow::Owned richiede obbligatoriamente che il dato passato al suo interno sia il tipo posseduto (Owned variant), che per le stringhe (str) è proprio il tipo String

il risultato è un dato che possiede la variabile: l'str che viene prodotto in uscita è posseduta, perchè l'operazione `to_uppercase()` può essere fatto solo con le stringhe e quindi il tipo di risultato deve essere di tipo **Owned**. Per poter possedere il dato si è dovuto creare una copia.

se non ho nessuna lettere minuscola, il codice non deve effettuare nessun cambiamento: prendo l'input che avevo e lo restituisco uguale —> in particolare posso restituirlo come riferimento incapsulandolo nella variante **Borrowed**. Non ho quindi l'esigenza di creare una nuova stringa su cui copiare il valore in input.

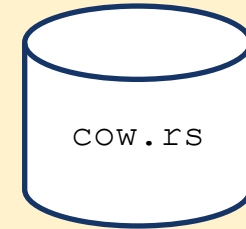
# std::borrow::Cow<'a, B>

- Il metodo **from()** crea un Cow da un riferimento a dati presi in prestito; Il compilatore, in base al tipo di dato fornito, colloca il valore nella variante Owned o Borrowed
- Il metodo **to\_mut()**
  - Se il Cow è Borrowed, **clona** i dati, trasforma se stesso in Owned e restituisce il riferimento mutabile;
  - se è già Owned, restituisce semplicemente il riferimento mutabile.
- Il metodo **into\_owned()**
  - se il Cow è Borrowed, clona i dati e viene restituita la proprietà;
  - se è Owned, il valore viene restituito senza copie aggiuntive.

```
use std::borrow::Cow;
```

54'00" 06-05-2026

```
fn modify_if_condition(content: Cow<str>) -> Cow<str> {
 if content.len() < 10 {
 Cow::Owned(content.into_owned().to_lowercase())
 } else {
 content
 }
}
```



nello scenario 1, `.into_owned()` non effettua alcuna copia nello Heap: spaccetta il Cow e restituisce il contenuto.

poichè qui stiamo passando una String, poichè il tipo String possiede i dati nello Heap, il compilatore genera un `Cow::Owned(String)`

```
fn main() {
 let long_string = String::from("This is a long string");
 let short_string = "SHORT";

 let modified_long = modify_if_condition(Cow::from(long_string));
 let modified_short = modify_if_condition(Cow::from(short_string));

 println!("{}", modified_long); // Stampa "This is a long string"
 println!("{}", modified_short); // Stampa "short"
}
```

al contrario, qui stiamo passando un `&str`; pertanto il compilatore assegna la variante `Borrowed`

Utilizzando il puntatore Cow, evitiamo il clonaggio quando non è necessario modificare i dati

nello scenario 1, `.into_owned()` non effettua alcuna copia nello Heap: spacchetta il Cow e restituisce il contenuto, perchè esso è già posseduto

nello scenario 2 Rust vede che `.into_owned()` lavora su un `&str`, per cui il dato è puntato. Per poter restituire un dato che detiene possesso, e deve essere così perchè viene inglobato nella variante `owned`, Rust deve clonare il dato e zilocarlo in una nuova zona dello heap

in entrambi i casi, `.into_owned()` restituisce un dato posseduto

```

use std::borrow::Cow;
fn abs_all<'a>(input: &'a mut Cow<'a, [i32]>) -> Cow<'a, [i32]>{
 // Check if the input slice contains any negative values
 if input.iter().any(|&x| x < 0) {
 // If yes, create a new vector with the absolute values
 let mut output = Vec::new();
 for &x in input.iter(){
 output.push(x.abs());
 }
 // Return the vector as an owned slice
 print!("Owned => ");
 Cow::Owned(output)
 } else {
 // If no, return the input slice as a borrowed slice
 print!("Borrowed => ");
 Cow::Borrowed(input)
 }
}

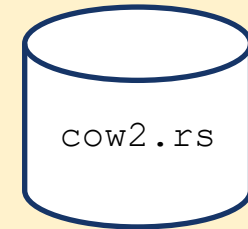
fn main() {
 let slice = [0, 1, 2];
 let mut input = Cow::from(&slice[..]);
 let mut output = abs_all(&mut input); // No clone occurs because input doesn't need to be mutated.
 println!("{:?}", output);

 let slice = [-1, 0, 1];
 let mut input = Cow::from(&slice[..]); // Clone occurs because input needs to be mutated.
 output = abs_all(&mut input);
 println!("{:?}", output);

 let mut input = Cow::from(vec![-1, 0, 1]); // No clone occurs because input is already owned.
 output = abs_all(&mut input);
 println!("{:?}", output);
}

```

se hai tempo guarda



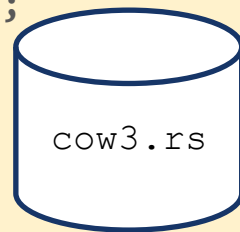
```
use std::borrow::Cow;
```

```
fn main() {
 let mut borrowed_data: Cow<str> = Cow::Borrowed("Salve Mondo!");

 let mut mutable_data = borrowed_data.to_mut();
 mutable_data.make_ascii_uppercase();

 println!("Mutated data: {}", mutable_data);
 println!("Original data: {}", borrowed_data);
}
```

se hai tempo guarda



Chiamiamo `to_mut()` su un'istanza `Cow<str>` presa in prestito.

Se `Cow<str>` è nella variante `Borrowed`, `to_mut()` lo converte nella variante `Owned`, consentendo l'accesso modificabile ai dati.

Otteniamo quindi un riferimento mutabile (`&mut str`) ai dati sottostanti ed eseguiamo operazioni mutabili su di esso.



# Smart pointer e metodi

- L'argomento **self** di un metodo può anche avere come tipo **Box<Self>**, **Rc<Self>**, o **Arc<Self>**
  - In tal caso, il metodo può essere solo invocato a partire dal corrispondente tipo di puntatore
  - L'invocazione del metodo passa la proprietà del puntatore al metodo stesso
- A differenza di quanto accade con i riferimenti, non è disponibile una forma abbreviata per la sintassi di self, il cui tipo deve essere dichiarato in modo esplicito, come nel caso dei parametri ordinari

```
struct Task {
 name: String,
}
impl Task {
 fn complete(self: Box<Self>) {
 println!("Task '{}' completed and dropped!", self.name);
 // self è consumato qui
 }
}
fn main() {
 let task = Box::new(Task { name: String::from("Studio degli Smart Pointer") });
 task.complete();
}
```

NOTAZIONE DA IMPARARE



i tipi Box o gli Rc possono essere inseriti nei metodi, i quali possono lavorare con un ricevitore di tipo Box piuttosto che Rc.

Tuttvia, diversamente dalle variabili tradizionali, ovvero quelle non incalspulate da uno smart pointer, dobbiamo riportare la notazione estesa `—> self: Box<Self>`

```
use std::rc::Rc;
```

```
struct Node {
 value: i32,
 next: Option<Rc<Node>>,
}
```

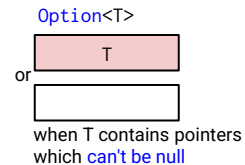
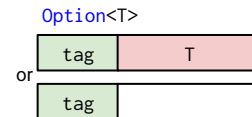
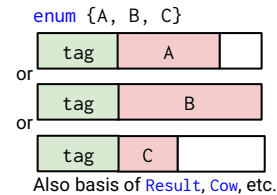
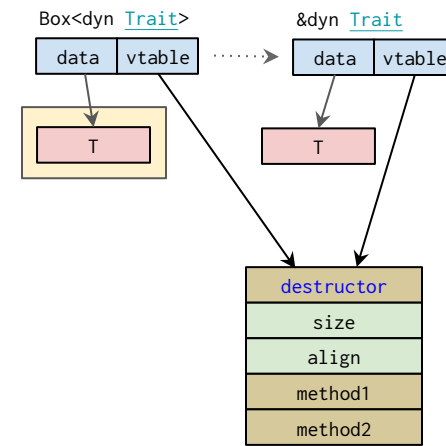
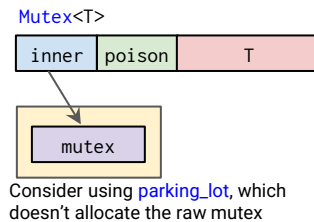
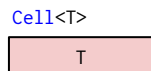
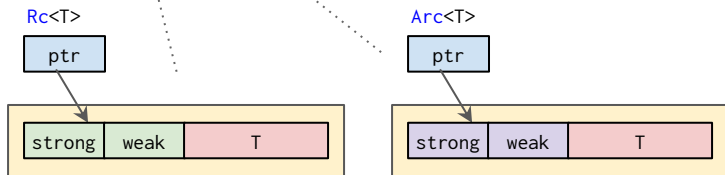
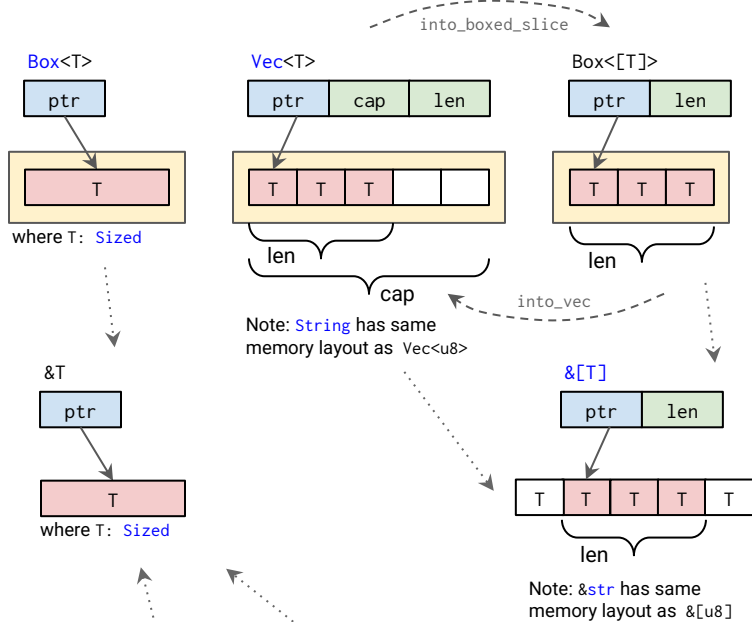
Ripassalo scrivendolo e meditandoci su vs code

```
impl Node {
 fn print_chain(self: Rc<Self>) {
 let mut current = Some(self);
 while let Some(node) = current {
 println!("{}", node.value);
 current = node.next.clone();
 }
 }
}

fn main() {
 let third = Rc::new(Node { value: 3, next: None });
 let second = Rc::new(Node { value: 2, next: Some(third) });
 let first = Rc::new(Node { value: 1, next: Some(second) });

 first.print_chain(); // Stampa 1, 2, 3
}
```





## Legend

ptr 4/8 bytes (usize)

size 4/8 bytes

atomic 4/8 bytes

fn 4/8 bytes

allocation heap allocation, implies ownership

T user defined type

deref

# Per saperne di più...

- Understanding smart pointers in Rust
  - <https://blog.logrocket.com/smart-pointers-rust/>
- Understanding Rust smart pointers
  - <https://medium.com/the-polyglot-programmer/undestanding-rust-smart-pointers-660d59715ab9>
- Rust Smart Pointers Tutorial
  - <https://www.koderhq.com/tutorial/rust/smart-pointer/>
- Smart Pointers in Rust: What, why and how?
  - <https://dev.to/robertorres/smart-pointers-in-rust-what-why-and-how-oma>

# Glossario

- Mutabilità interna: possibilità di modificare un campo all'interno di una struttura dichiarata immutabile (immutabilità esterna)